Understanding Git Concepts

Mental Model Before Commands

Build the right mental picture first — commands come later.

M. Souhil | Dev Web Core Team

Data Science Club — USTHB

Tech Department | Dev Web Section

March 8, 2026 | Concepts Session

Session Roadmap

1. **History of Git** — Origin story
2. **Why Version Control?** — The problem it solves
3. **Git Core Mental Model** — Snapshots, areas, branches, DAG
4. **Git vs GitHub** — Local vs cloud

1. **Collaboration Concepts** — Remote, push, pull
2. **Linux vs Windows** — Platform differences
3. **Workflow Overview** — Edit → Stage → Commit → Push
4. **Mini-Challenge** — Quiz & DAG drawing

Note

No terminal commands today. We are building the **mental model** that makes every command make sense.

Part 0 — A Brief History of Git

Where Did Git Come From?

- **2005** — Git is created
- By **Linus Torvalds** — creator of the Linux kernel
- **Trigger:** BitKeeper (the previous tool) **revoked its free license** for the Linux project
- Torvalds needed a new tool — and built one in **10 days**
- Designed from day one for:
  - **Speed** — handle the Linux kernel's scale
  - **Distribution** — every developer has the full history
  - **Data integrity** — every object is verified by hash
  - **Non-linear development** — thousands of parallel branches

> *I'm an egotistical bastard, and I name all my projects after myself. First Linux, now Git.*
>
> — Linus Torvalds

Note

git = British slang for an unpleasant person. Torvalds named it after himself intentionally.

Part 1 — Why Version Control Exists

The Problem Every Developer Faces

Have you ever done any of these?

- Deleted code that worked, then couldn't get it back
- Sent a file called `project_final_v2_REAL.zip`
- Two teammates edited the same file — one overwrote the other
- "It worked yesterday — what changed?!"
- Wanted to try a risky change but feared breaking everything
- No idea which version of the file is the latest one

A real developer's folder

```
project.zip
project_v2.zip
project_final.zip
project_final_2.zip
project_REAL_final.zip
project_USE_THIS_ONE.zip
project_no_really_this.zip
```

What Version Control Actually Does

Key Concept

A **Version Control System (VCS)** records every change to your files over time, so you can recall any specific version at any point.

It gives you:

- **History** — go back to any past state
- **Branching** — try risky things safely
- **Collaboration** — multiple people, no overwriting
- **Accountability** — who changed what and when
- **Safety net** — nothing is ever truly lost

Analogy 1: Google Docs

Google Docs has **Version History**. Click it and you see every change ever made, by whom, and when. You can restore any past version in one click.

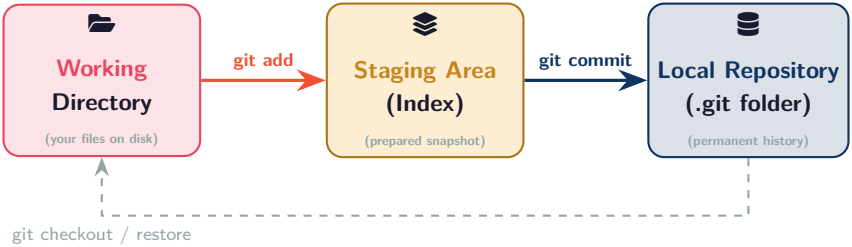
Git does exactly this — but for **code**, and it works **offline**.

Analogy 2: Game Saves

Every Git **commit** is like a **game save checkpoint**. You can always load a previous save if you mess up the current level.

Part 2 — Git Core Mental Model

The Three Areas of Git



Working Directory
The files you see and edit. Any unsaved changes live here.

Staging Area
A **draft zone**. You choose exactly which changes go into the next commit.

Local Repository
The permanent database of all your commits — stored inside the `.git/` folder.

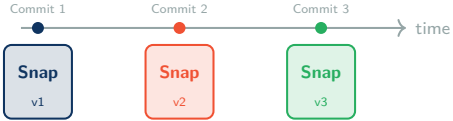
What Is a Snapshot?

Key Concept

When you **commit**, Git does not store the **diff** (line-by-line changes). It stores a complete **snapshot** — the full state of every tracked file at that exact moment.

Why snapshots, not diffs?

- **Speed:** restore any version instantly — no replaying diffs
- **Integrity:** the entire state is verified by a single hash
- **Completeness:** you always know exactly what the project looked like



Analogy

Every commit = a **photo of your entire project** at one moment. The whole Git history = a photo album you can flip through freely.

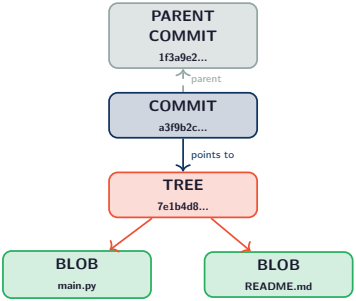
Note

Optimization: If a file has not changed between two commits, Git does **not** store it twice — it stores a **pointer** to the same data. Space is not wasted.

How Git Stores Data — The Object Model

Inside `.git/objects/`, Git stores:

- **Blob** — contents of a single file
- **Tree** — a directory listing (file names + blob refs)
- **Commit** — a snapshot reference + metadata
- **Tag** — a named pointer to a commit



Key Concept

Every object gets a **SHA-1 hash** — a 40-character unique fingerprint based entirely on the **content**. Same content = same hash. Any change = completely different hash.

Note

SHA-1 → SHA-256: Git already supports SHA-256 (opt-in). The switch is coming because SHA-1 has known **theoretical collision vulnerabilities** — two different inputs producing the same hash. SHA-256 is **cryptographically stronger** and future-proof. Your daily workflow does not change.

What Exactly Is a Commit?

Key Concept

A **commit** is a permanent snapshot stored in the repository. It contains:

- **A snapshot** — pointer to the root Tree object
- **A unique hash** — `a3f9b2c7...`
- **Parent pointer(s)** — link to previous commit
- **Author + email + timestamp**
- **Commit message** — your description of the change

What a commit looks like internally

```
commit a3f9b2c7d...  tree
7e1b4d8f3...  parent 1f3a9e2b1...
author M. Souhil <mail@usthb.dz>
2025-03-01 14:30:21 committer M.
Souhil 2025-03-01 14:30:21
Add homepage navigation bar
```

Analogy

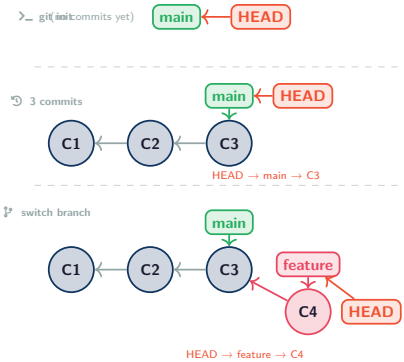
A commit is like **sending a letter**. Once posted, you cannot un-send it. The post office (Git) keeps a record forever.

What Is a Branch?

Key Concept

A **branch** is simply a **named pointer** to a commit. Nothing more. It is not a copy of the code — it is just a label.

- **HEAD** exists from `git init` — initially points to `main` (no commits yet)
- Each new commit: branch pointer **moves forward automatically**
- `git switch feature` → HEAD follows — now points to `feature`
- New branch = a new label only — **no files copied!**



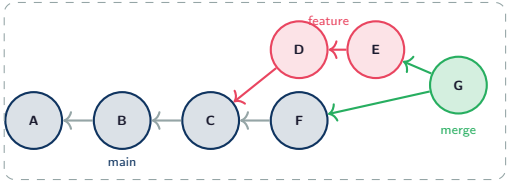
Analogy

A branch is like a **sticky note** attached to a commit. Moving the sticky note costs nothing. The commit itself never moves.

The Directed Acyclic Graph (DAG)

Key Concept

Git's commit history is a **Directed Acyclic Graph (DAG)**.



Acyclic: arrows never loop back

What does that mean?

- **Directed** — every commit points **to its parent** (arrows always go backward in time)
- **Acyclic** — you can **never** follow arrows back to a commit you already visited. No loops possible.
- **Graph** — commits can have **multiple children** (branching) or **multiple parents** (merging)

Note

The DAG structure is why Git can always reconstruct any version of the project efficiently and safely — there is a clear, unambiguous path through history.

Merge Type 1 — Fast-Forward ⏩

Key Concept

Condition: `main` has **not moved** since `feature` branched off.

Git simply **slides the pointer forward** — no new commit is created.

Before merge:

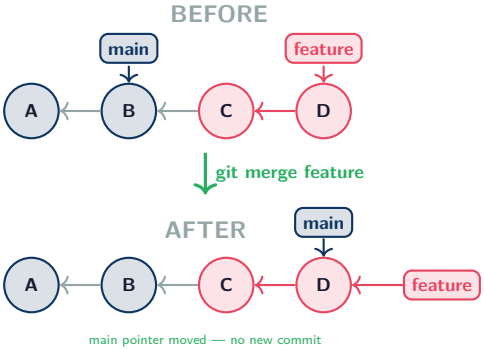
`main` is stuck at B.

`feature` has 2 new commits (C, D).

After merge:

`main` pointer **jumps to D**.

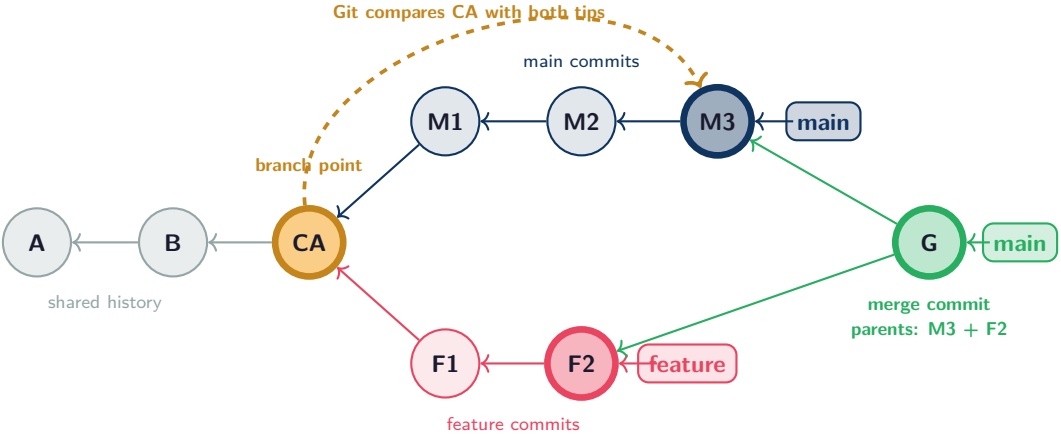
History is a clean straight line.



Note

The history looks as if you never branched — one straight line of commits. Clean & simple.

Merge Type 2 — Three-Way Merge: The Graph



Merge Type 2 — How Git Decides: The 3-Snapshot Rule

[1] CA	[2] main tip	[3] feature tip	Git Result
line 3 `color=blue`	`color=red`	`color=blue` — only main ≠ CA ⇒ no conflict, auto-pick main	pick red
line 7 `size=12`	`size=12`	`size=16` — only feature ≠ CA ⇒ no conflict, auto-pick feature	pick 16
line 11 `msg=Hi`	`msg=Hello`	`msg=Hey` — both ≠ CA, different values ⇒ Git pauses, you decide	CONFLICT

Key Concept

The rule Git uses per line/file:

Only one side changed vs CA ⇒ auto-merge, no conflict.

Both sides changed differently ⇒ conflict, Git pauses.

Neither side changed ⇒ keep CA value.

Note

CA is found **automatically** — Git walks both commit chains backward until they meet.

You never choose CA manually.

Merge Conflicts — What Can Conflict?

Type 1 — Same lines edited differently

```
Ancestor:  color = blue
```

→ CONFLICT

```
main:    color = red
```

```
feature: color = green
```

Git-auto-merges: different lines touched by each branch.

Git pauses: same line, two different values.

Type 2 — File renamed on one branch, edited on other

```
main:    renamed app.py → server.py
feature:  edited line 12 of app.py
```

Git flags: file identity ambiguous

Type 3 — One branch deletes, other edits

```
main:    deleted config.py
feature:  modified config.py line 5
```

Git asks: keep the file or delete it?

Type 4 — Both branches add same filename

```
main:    created utils.py (version A)
feature:  created utils.py (version B)
```

Git cannot pick — both are "new"

Important

In **all cases**: Git **pauses the merge**, marks the conflicting files, and waits.
It **never silently overwrites** your work.

Resolution covered in the next session.

Part 3 — Git vs GitHub

Git Is Not GitHub

Git ≠ GitHub

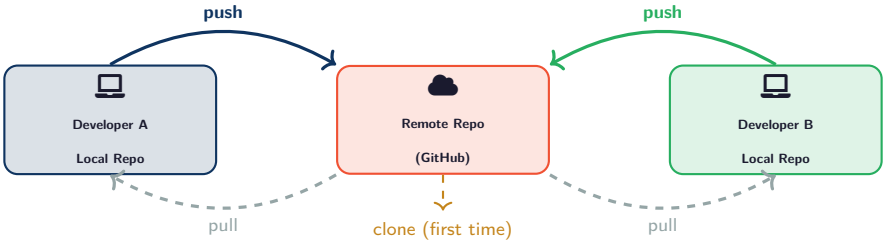
	Git	GitHub
What it is	A version control tool (software)	A cloud hosting platform (website)
Where it runs	On **your computer** — local	On **Microsoft's servers** — remote
Internet needed?	**No** — works 100% offline	**Yes** — requires internet
Who made it	Linus Torvalds, 2005	Tom Preston-Werner et al., 2008
What it adds	Commit, branch, merge, history	Collaboration, pull requests, CI/CD, web UI
Alternatives	Mercurial, SVN, Fossil	GitLab, Bitbucket, Gitea

Key Concept

Simple rule: **Git** = the engine under the hood. **GitHub** = the cloud garage where you park and share it.
You can use Git without GitHub. You cannot use GitHub without Git.

Part 4 — Basic Collaboration Concepts

How Collaboration Works



Clone
Copy a remote repo to your machine for the first time

Push
Send your local commits to the remote

Pull
Get new commits from remote into your local repo

Pull Request
Ask teammates to review & merge your branch

Can We Collaborate Without GitHub? ?

Key Concept

Yes! GitHub is a *convenience*, not a requirement. Git only needs a network path to a repository.

Alternative collaboration methods:

- **SSH** — push/pull over a local network to another machine's Git repo
- **Shared folder** — a bare Git repository on a shared drive (USB, network drive)
- **Self-hosted** — run your own GitLab/Gitea server on a university LAN
- **Email patches** — the original Linux kernel workflow (still used by some projects!)

The key insight:

A **remote** in Git is just a **URL or path**.

`https://github.com/user/repo`

`ssh://user@192.168.1.5/repo`

`/mnt/shared-drive/repo.git`

All three are valid Git remotes.

Part 5 — Linux vs Windows Differences

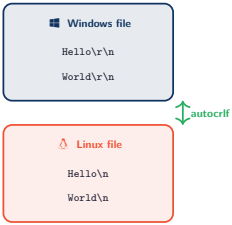
Git on Linux vs Windows vs

	Linux / macOS	Windows
Git pre-installed?	Often yes (or 1 command away: `apt install git`)	**No.** Must install Git for Windows
Shell	Bash natively	PowerShell / CMD (limited); Git Bash provided
File system case	**Case-sensitive** (`File.txt` ≠ `file.txt`)	Case-insensitive (`File.txt` = `file.txt`)
Line endings	LF (`\n`)	CRLF (`\r\n`)
Executable bit	Tracked by Git	Ignored (no Unix perms)
Performance	Native — very fast	Slightly slower (POSIX layer)
WSL available?	N/A	**Yes** — Windows Subsystem for Linux

Line Endings — The Invisible Problem

What is the problem?

- Every line of text ends with an invisible character
- **Linux/macOS:** uses `LF` (Line Feed, `\n`)
- **Windows:** uses `CRLF` (Carriage Return + Line Feed, `\r\n`)
- If not handled, Git sees **every single line** as "changed" when switching between OS
- This pollutes the entire git history with meaningless changes



Note

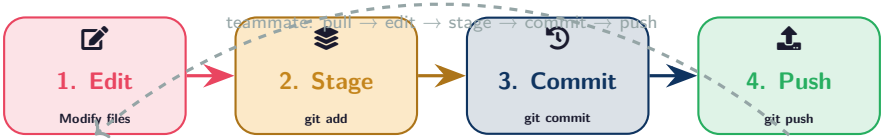
WSL (Windows Subsystem for Linux) runs a full Linux environment inside Windows. It eliminates most Git compatibility issues and is recommended for serious development on Windows.

Key Concept

Solution: set `core.autocrlf` in Git config.
Windows users: `core.autocrlf = true` (convert on checkout, convert back on commit)
Linux/macOS: `core.autocrlf = input` (convert CRLF to LF on commit only)

Part 6 — Basic Workflow Overview

The Git Workflow in One Picture



Edit
Write or change code in your Working Directory

Stage
Choose exactly which changes to include in the next commit

Commit
Save a permanent snapshot to your local repository

Push
Upload your commits to the remote (GitHub) for others to see

Part 7 — Mini Interactive Challenge

Challenge Time!

Let's Check Your Mental Model!

3 Challenges:

Challenge 1 — Conceptual Quiz (5 MCQ)

Challenge 2 — Draw the DAG

Challenge 3 — Local Activity (Install & Init)

Challenge 1 — Conceptual Quiz ?

Question	Options
1. What does the Staging Area do?	(a) Backup to internet **(b) Prepare changes for next commit** (c) Store full history
2. A branch in Git is:	(a) A copy of the project folder **(b) A pointer to a commit** (c) A backup on GitHub
3. What is a merge conflict?	(a) A network error (b) Git refusing to commit **(c) Two branches edited the same lines**
4. Git requires GitHub to work.	(a) True **(b) False — Git works offline**
5. What is HEAD in Git?	(a) The first commit (b) The server address **(c) Pointer to the current branch**

Answers: b, b, c, b, c Score 5/5? You have a solid Git mental model!

Challenge 2 — Draw the DAG

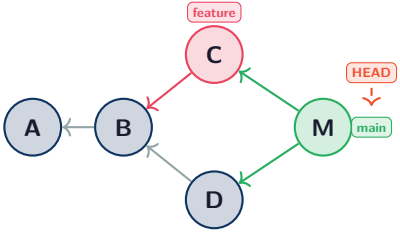
Given these actions in order:

1. Make commit **A** on branch `main`
2. Make commit **B** on branch `main`
3. Create new branch `feature` at **B**
4. Make commit **C** on `feature`
5. Switch back to `main`, make commit **D**
6. Merge `feature` into `main` → commit **M**

Questions:

- ? Draw the DAG. Where do `main` and `feature` point after the merge?
- ? How many parents does commit **M** have?
- ? Where does **HEAD** point at the end?

Answer:



M has 2 parents: C and D

Challenge 3 — Local Activity (Take-Home) >_

Steps (concept, no deep CLI):

1. **Install Git** on your machine
   - Linux: `sudo apt install git`
   - Windows: download `git-scm.com`
2. **Create a folder** called `my-first-repo`
3. **Initialize** Git inside it (`git init`) → This creates the hidden `.git/` folder
4. **Create a file** (e.g. `hello.txt`) and write something in it
5. **Stage it** (`git add hello.txt`) → Moves it to the Staging Area
6. **Commit** it (`git commit -m "first commit"`) → Creates your first snapshot
7. **Make a change**, stage and commit again → Now you have a 2-commit history!

Note

This is a **guided conceptual activity** for after the session. Follow each step mentally before trying the terminal.

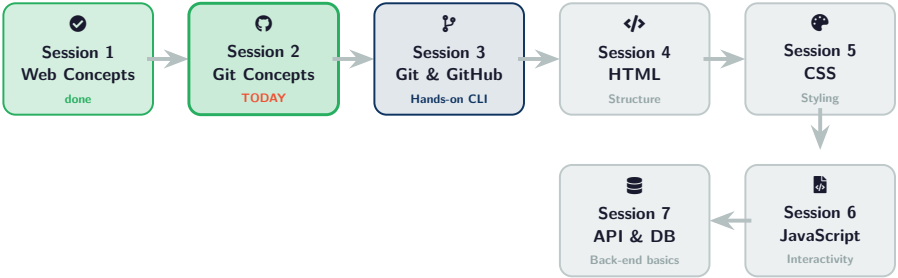
After doing this, you will have:

- A real `.git/` folder on your disk
- Two commits in your local repository
- A working mental model you can **feel**

Note

CLI commands will be covered fully in the **next session**. Today: just run them and observe what happens.

What's Coming Next — Full Track Roadmap



Before Next Session (Git CLI)

- Do the **local activity** from Challenge 3
- Install **Git** on your machine
- Create a **GitHub account**
- Run `git init` — make 2 commits

Git Resources

- `git-scm.com/book` — Pro Git (free)
- `learngitbranching.js.org` — visual
- `ohshitgit.com` — fix mistakes
- `docs.github.com` — GitHub guides

You Have the Map.

The commands will make sense now.

Next session: hands-on CLI — init, add, commit, branch, merge.

Mental models you now own:

- Working Dir → Stage → Repo
- Commit = snapshot + hash + parent
- Branch = pointer, not a copy
- DAG = directed, acyclic, no loops
- Git ≠ GitHub

mokeddemsouhil968@gmail.com

datascienceclubusthb-club

Data Science Club — USTHB

Tech Dept | Dev Web Section

Appendix — Key Terms Glossary

Term	Definition
`repository`	The `.git/` database of your project history
`commit`	A permanent snapshot with hash + parent
`branch`	A named pointer to a commit
`HEAD`	Pointer to the current branch
`merge`	Combine two branches into one
`staging`	The index / preparation zone for commits
`blob`	Git object storing a file's content
`tree`	Git object storing a directory listing
`hash`	40-char fingerprint of a Git object

Term	Definition
`clone`	Download a full remote repo locally
`push`	Upload local commits to remote
`pull`	Download + merge remote commits locally
`remote`	A URL pointing to another repository
`PR`	Pull Request — review + merge request
`DAG`	Directed Acyclic Graph — Git's history structure
`LF`	Unix line ending (`\n`)
`CRLF`	Windows line ending (`\r\n`)
`WSL`	Windows Subsystem for Linux

Appendix — Platforms Comparison

Platform	Owned by	Free tier	Self-host	Best for
GitHub	Microsoft	**Yes**	No (GitHub Enterprise)	Open source, portfolios
GitLab	GitLab Inc.	**Yes**	**Yes (free)**	CI/CD, self-hosted teams
Bitbucket	Atlassian	**Limited**	No	Jira-integrated teams
Gitea	Community	**Yes**	**Yes**	Lightweight self-hosting
Forgejo	Community	**Yes**	**Yes**	Privacy-first fork of Gitea

Key Concept

For this club: **GitHub** for open collaboration and portfolio visibility. For advanced projects or university infrastructure: consider **GitLab self-hosted**.